

JDBC

(Java Database Connectivity)

Dr. H. M. Singh

Department of Computer Science and IT

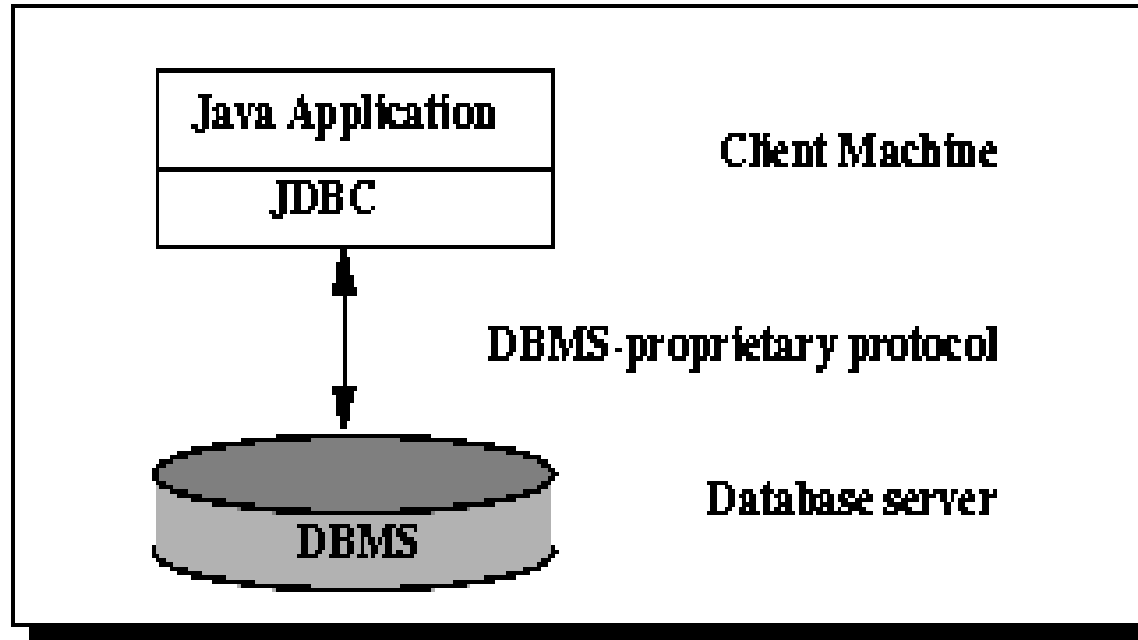
ODBC (Open Database Connectivity)

- A standard or open application programming interface (API) for accessing a database.
- By using ODBC statements in a program, you can access files in a number of different databases, including Access, dBase, DB2 and Excel
- It allows programs to use SQL requests that will access databases without having to know the proprietary interfaces to the databases.
- ODBC handles the SQL request and converts it into a request the individual database system understands.

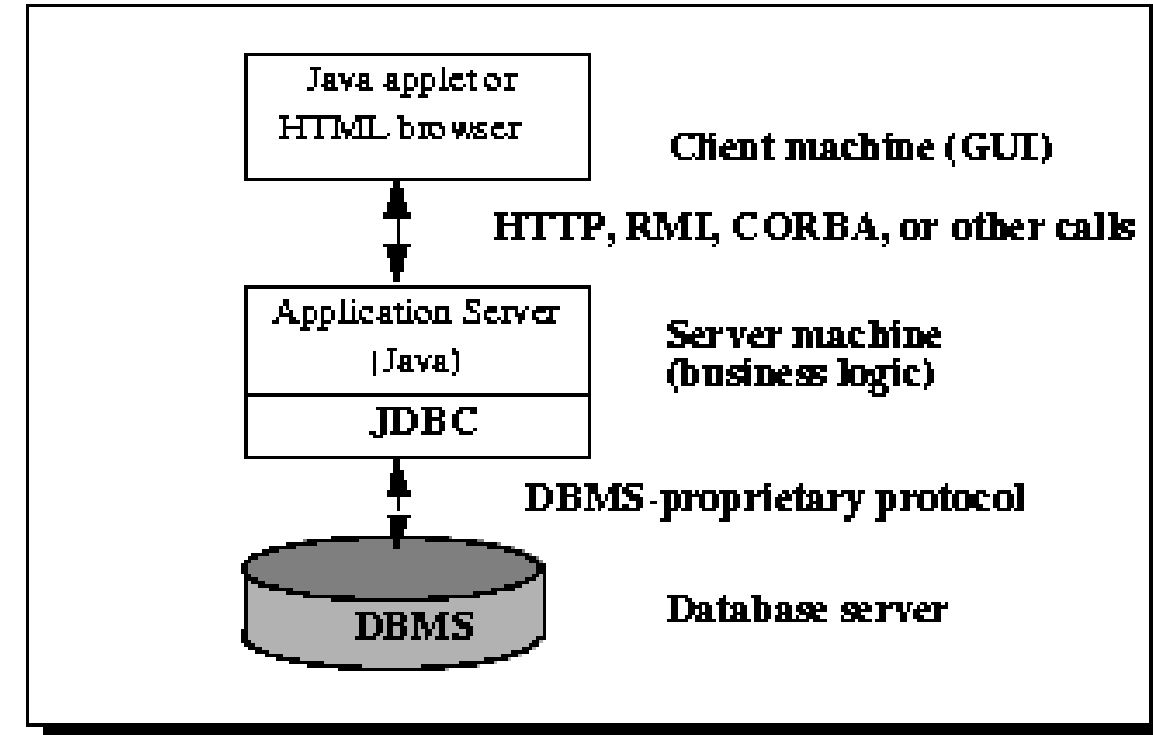
JDBC (Java Database Connectivity)

- JDBC
 - is a Java API for connecting programs written in Java to the data in relational databases.
 - consists of a set of classes and interfaces written in the Java programming language.
 - provides a standard API for tool/database developers and makes it possible to write database applications using a pure Java API.
 - The standard defined by Sun Microsystems, allowing individual providers to implement and extend the standard with their own JDBC drivers.
- JDBC helps you to write Java applications that manage these three programming activities:
 - Connect to a data source, like a database
 - Send queries and update statements to the database
 - Retrieve and process the results received from the database in answer to your query

JDBC Architecture



Two-tier Architecture for Data Access



Three-tier Architecture for Data Access.

JDBC Components

- The JDBC API

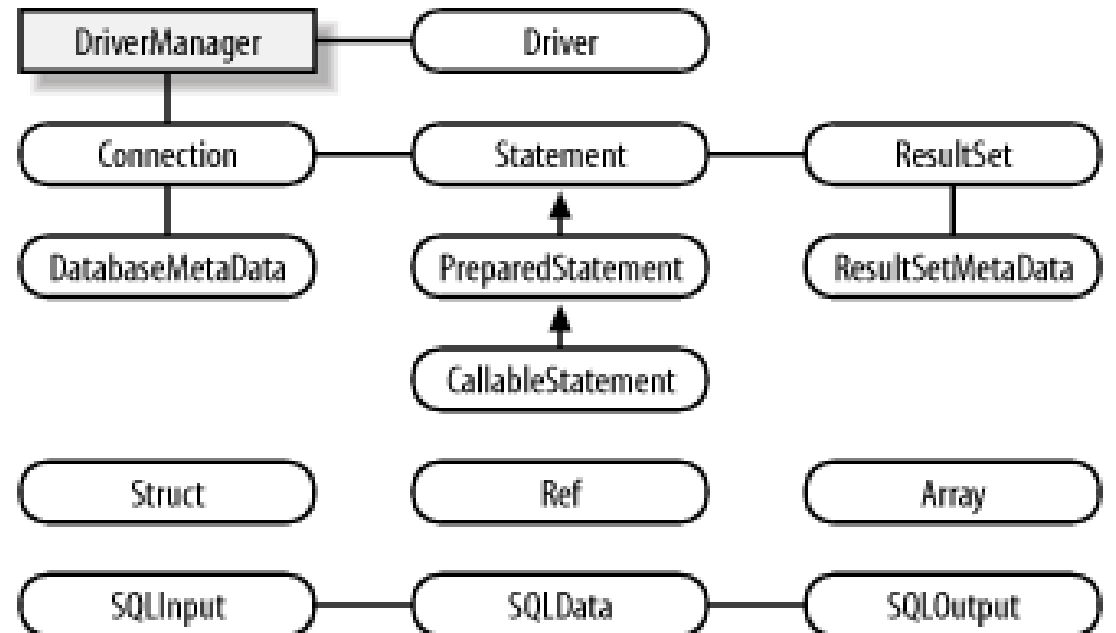
- provides programmatic access to relational data from the Java programming language. Using the JDBC API, applications can execute SQL statements, retrieve results, and propagate changes back to an underlying data source.
- The JDBC API can also interact with multiple data sources in a distributed, heterogeneous environment.
- The JDBC API is part of the Java platform, which includes the Java Standard Edition (Java SE) and the Java Enterprise Edition (Java EE).

- JDBC Driver Manager

- The JDBC **DriverManager** class defines objects which can connect Java applications to a JDBC driver. DriverManager has traditionally been the backbone of the JDBC architecture
- The Standard Extension packages javax.naming and javax.sql let you use a DataSource object registered with a Java Naming and Directory Interface™ (JNDI) naming service to establish a connection with a **data source**.

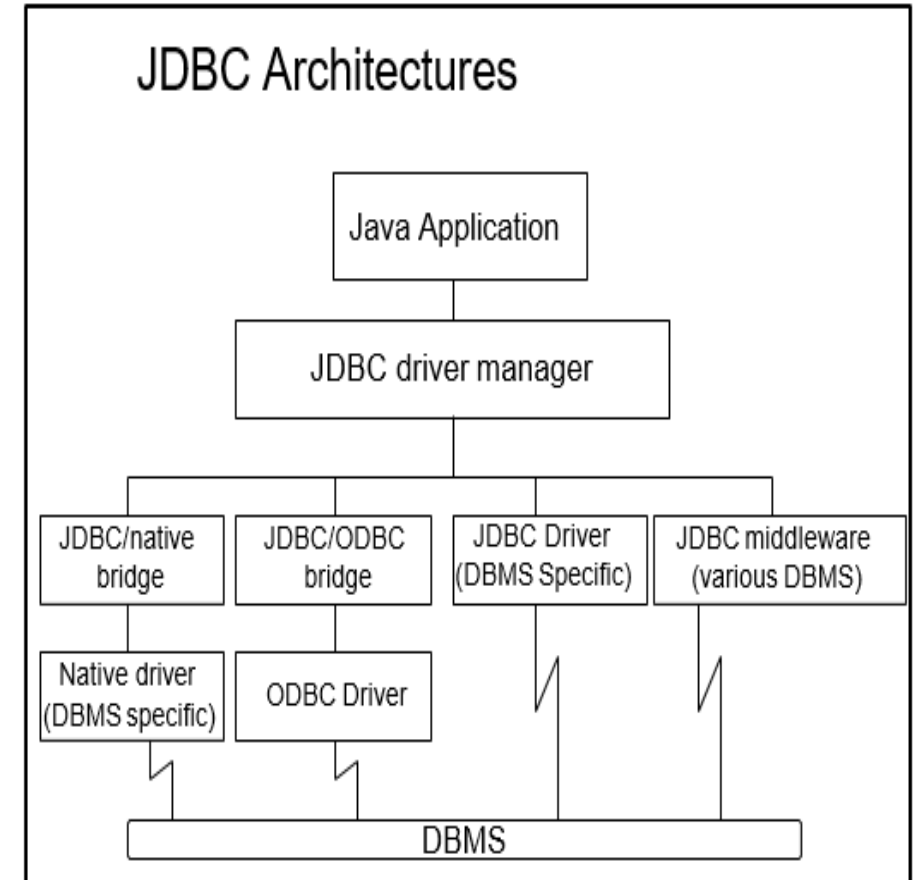
The JDBC API

- The JDBC API is based mainly on a set of interfaces, not classes.
- It's up to the manufacturer of the driver to implement the interfaces with their own set of classes.
- Notice that the only concrete class is DriverManager. The rest of the core API is a set of interfaces.



JDBC Drivers

- A JDBC driver is set of Java classes that implement JDBC interfaces for interacting with a specific database.
- Almost all database vendors such as MySQL, Oracle, Microsoft SQL Server, provide JDBC drivers.
- There are three/four types of JDBC drivers
 - JDBC-ODBC bridge
 - JDBC-native API Driver,
 - JDBC-net Driver,
 - JDBC Driver.



JDBC-ODBC Bridge Driver

- The JDBC-ODBC bridge driver uses ODBC driver to connect to the database. The JDBC-ODBC bridge driver converts JDBC method calls into the ODBC function calls.

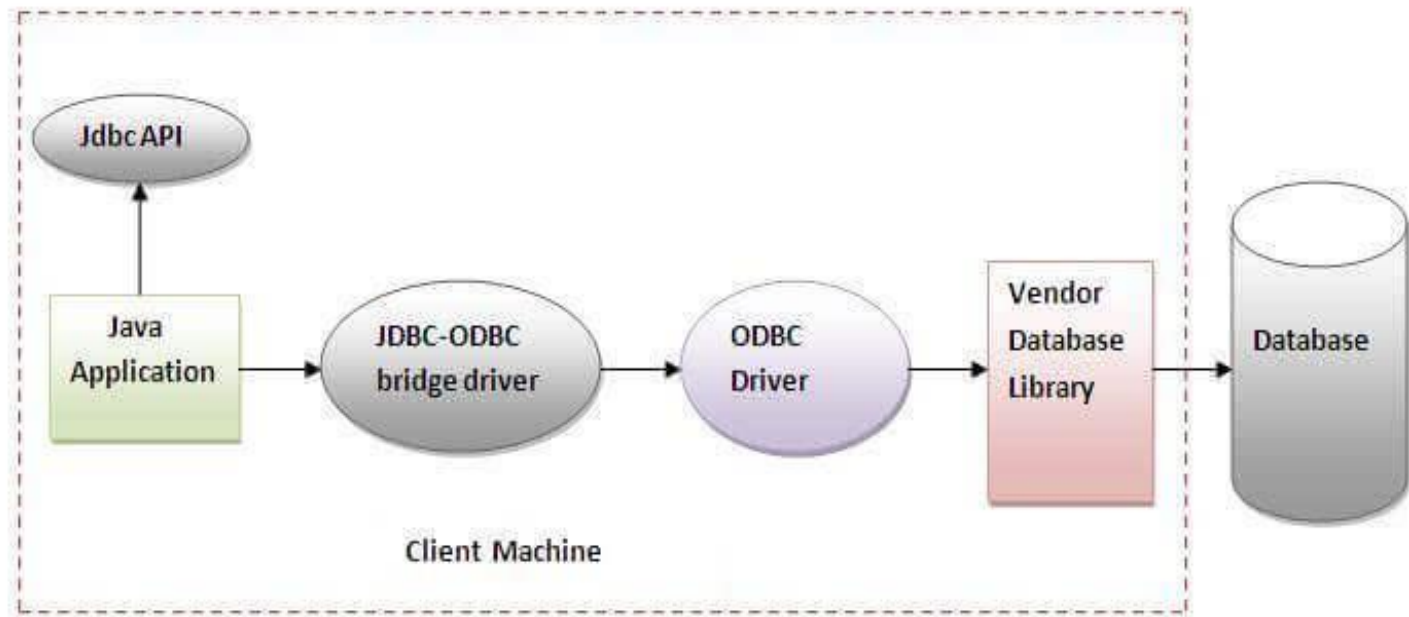


Figure- JDBC-ODBC Bridge Driver

Native-API driver (Partially Java)

- Native API drivers means that the driver contains Java code that calls native C or C++ methods provided by the individual database vendors that perform the database access.
- Again, this solution requires software on the client system. It is not written entirely in java.

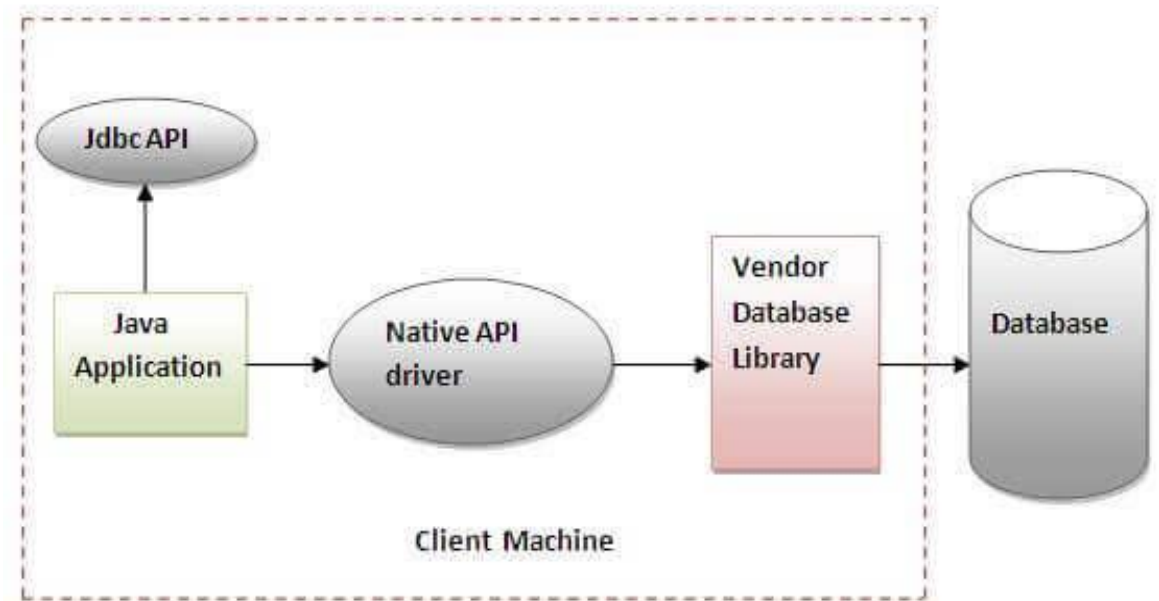
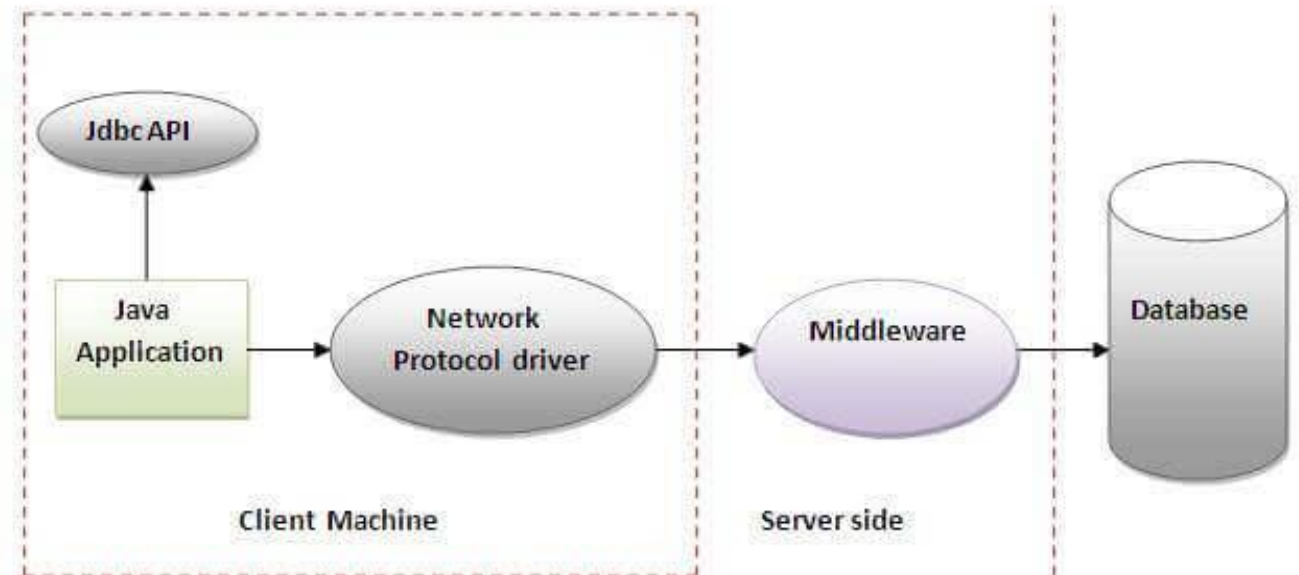


Figure- Native API Driver

Network Protocol driver (middleware driver)

- JDBC driver(generic) on the client uses sockets to call a middleware application on the server that translates the client requests into an API specific to the desired driver.
- This kind of driver is extremely flexible, since it requires no code installed on the client and a single driver can actually provide access to multiple databases.
- It is fully written in java.



Thin driver (Pure Java Driver)

- The thin driver converts JDBC calls directly into the vendor-specific database protocol. That is why it is known as thin driver. It is fully written in Java language.

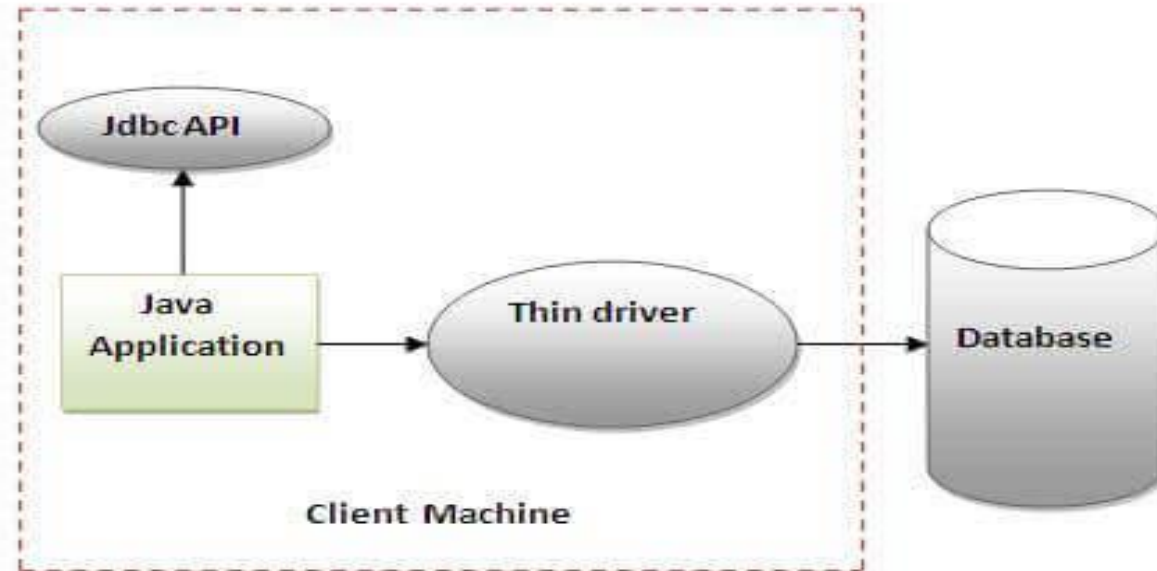


Figure- Thin Driver

JDBC Steps

1. Importing Packages
2. Registering the JDBC Drivers
3. Opening a Connection to a Database
4. Creating a Statement Object
5. Executing a Query and Returning a Result Set Object
6. Processing the Result Set
7. Closing the Result Set and Statement Objects
8. Closing the Connection

JDBC Steps

- Importing Packages

```
import java.sql.*;
```

- Registering/Loading the JDBC Drivers

```
DriverManager.registerDriver (neworacle.jdbc.driver.OracleDriver());
```

- Opening a Connection to a Database

```
Connection con = DriverManager.getConnection (  
    "jdbc:oracle:thin:@aardvark:1526:teach ", user, password);
```

Creating a Statement Object

```
Statement stmt = con.createStatement();
```

JDBC Steps

- Executing a Query and Returning a Result Set Object

```
ResultSet rs = stmt.executeQuery("Select * from student");
```

- Processing the Result Set

```
while(rs.next) {  
    -----  
}
```

- Closing the Result Set and Statement Objects

```
rs.close()  
stmt.close()
```

Closing the Connection

```
con.close();
```

Simple JDBC Program

```
import java.sql.*;
class TestJdbc{
    public static void main(String args[]) {
        try{
            Class.forName("net.ucanaccess.jdbc.UcanaccessDriver");
            String msAccDB = "C:/Users/adity/Desktop/betchcsevi.accdb";
            String dbURL = "jdbc:ucanaccess://"+ msAccDB;
            Connection con=DriverManager.getConnection(dbURL);
            Statement stmt=con.createStatement();
            ResultSet rs=stmt.executeQuery("select * from student");
            while(rs.next()){
                System.out.println(rs.getInt(1)+" "+rs.getString(2)+" "+rs.getInt(3));
            }
            con.close();
        }catch(Exception e){ System.out.println(e);}
    }
}
```

Difference between Java 1.7 and Java 1.8 versions ?

- Until **Java 1.7** version, we are using Jdbc-Odbc bridge to connect MS Access database using the JDBC driver class

sun.jdbc.odbc.JdbcOdbcDriver

- Whereas in **Java 1.8** version, ucanaccess driver should be used to connect to MS Access database using driver class

net.ucanaccess.jdbc.UcanaccessDriver

Data Types

- There are data types specified to SQL that need to be mapped to Java data types if the user expects Java to be able to handle them.
- Conversion falls into three categories:
 - SQL type to Java direct equivalents
 - SQL INTEGER direct equivalent of Java int data type.
 - SQL type can be converted to a Java equivalent.
 - SQL CHAR, VARCHAR, and LONGVARCHAR can all be converted to the Java String data type.
 - SQL data type is unique and requires a special Java data class object to be created specifically for their SQL equivalent.
 - SQL DATE converted to the Java Date object that is defined in java.Date especially for this purpose.